

Université Ibn Tofaïl
Faculté des Sciences de Kénitra
Master Intelligence Artificielle et Objets Connectés

Rapport de Projet Machine Learning

Prédiction de victoire dans League of Legends à partir des statistiques à 15 minutes

Projet basé sur les données professionnelles 2024 d'Oracle's Elixir

**Réalisé par : Wassim Trachli Yassine zaimi Alae Bajoudi
Mouad Semsam Bouchaïb Salianni**

Encadré par : Said Ammar

Khadija Lakdioui

Module : Machine Learning

Année universitaire : 2025–2026

Juin 2026

Remerciements

Nous tenons tout d'abord à exprimer nos sincères remerciements à notre encadrant, **M. Said Ammar**, pour son accompagnement, ses orientations et ses conseils tout au long de la réalisation de ce projet.

Nous remercions également l'ensemble des enseignants et responsables de notre formation pour la qualité de l'encadrement pédagogique, ainsi que pour les connaissances acquises dans les domaines de l'intelligence artificielle, de l'analyse de données et du Machine Learning.

Ce projet nous a permis de mettre en pratique plusieurs notions importantes, notamment le prétraitement des données, la construction de variables pertinentes, l'entraînement de modèles de Machine Learning, l'évaluation des performances et le déploiement d'une application de prédiction. Il nous a également permis de mieux comprendre les exigences d'un projet appliqué, depuis l'exploitation d'un dataset réel jusqu'à l'interprétation des résultats obtenus.

Enfin, nous remercions l'ensemble des membres du groupe pour leur implication, leur sérieux et leur contribution à la réalisation de ce travail.

Yassine Zaimi, Wassim Trachli, Alae Bajoudi, Mouad Semsar, Salianni Bouchaïb

Résumé

Ce rapport présente un projet de Machine Learning visant à prédire l'issue d'un match professionnel de *League of Legends* à partir de statistiques disponibles autour de la quinzième minute de jeu. Le projet utilise un dataset issu d'Oracle's Elixir, contenant des données détaillées sur des matchs professionnels de la saison 2024. L'objectif principal est de construire un modèle de classification binaire capable de prédire si une équipe va gagner ou perdre, en se basant sur des variables telles que la différence d'or, la différence d'expérience, les plaques de tourelles, le KDA collectif, l'avantage de draft, le côté de la carte et les premiers objectifs obtenus.

La démarche suivie couvre l'ensemble du cycle d'un projet Machine Learning : compréhension du contexte métier, analyse du dataset, prétraitement des données, feature engineering, analyse exploratoire, entraînement de plusieurs modèles, comparaison des performances, optimisation, création de modèles d'ensemble et déploiement dans une application web Flask. Plusieurs algorithmes sont comparés, notamment la régression logistique, Random Forest, Gradient Boosting, AdaBoost, XGBoost, Voting Classifier et Stacking Classifier.

Les résultats expérimentaux montrent que les meilleurs modèles atteignent environ 77% à 78% d'accuracy dans le cadre expérimental utilisé. Les variables les plus importantes sont la différence d'or à quinze minutes, la différence d'expérience et l'avantage de draft. Le projet aboutit également à une application Flask permettant de saisir les informations d'un match et d'obtenir une prédiction accompagnée d'un niveau de confiance.

Le rapport discute enfin les limites du projet, notamment le risque de fuite d'information lié à la séparation train/test par lignes d'équipes, la possible utilisation d'objectifs obtenus après quinze minutes, l'approximation de l'expérience dans l'application et la simplicité du calcul de l'avantage de draft. Des pistes d'amélioration sont proposées afin de rendre le système plus robuste, plus fiable et plus proche d'une utilisation réelle.

Mots-clés : Machine Learning, classification binaire, League of Legends, esport, Oracle's Elixir, Flask, prédiction de victoire, Random Forest, XGBoost, Voting Classifier.

Abstract

This report presents a Machine Learning project designed to predict the outcome of a professional *League of Legends* match using statistics available around the fifteenth minute of gameplay. The project relies on a dataset from Oracle's Elixir containing detailed information about professional matches from the 2024 competitive season. The main objective is to build a binary classification model able to predict whether a selected team will win or lose based on game-state variables such as gold difference, experience difference, turret plate difference, collective KDA, draft advantage, side selection and early objectives.

The work follows a complete Machine Learning pipeline : business context analysis, dataset understanding, preprocessing, feature engineering, exploratory data analysis, model training, model comparison, hyperparameter tuning, ensemble learning and deployment through a Flask web application. Several models are compared, including Logistic Regression, Random Forest, Gradient Boosting, AdaBoost, XGBoost, Voting Classifier and Stacking Classifier.

The experimental results show that the best models reach around 77% to 78% accuracy under the current experimental setup. The most influential variables are gold difference at fifteen minutes, experience difference and draft advantage. The project also includes a Flask application that allows users to input match information and obtain a win/loss prediction with a confidence score.

The report concludes with a critical discussion of the project's limitations, including potential data leakage due to row-level train/test splitting, the possible use of post-15-minute objective information, the approximation of experience difference in the application and the simplified draft advantage calculation. Several improvements are proposed to make the system more reliable, robust and suitable for real-world use.

Table des matières

Résumé	ii
Abstract	iii
Guide d'insertion des figures	x
1 Introduction générale	1
2 Problématique et objectifs du projet	4
2.1 Contexte de la problématique	4
2.2 Nature du problème Machine Learning	4
2.3 Objectif général	5
2.4 Objectifs spécifiques	5
2.5 Intérêt du choix de la quinzième minute	6
3 Présentation du jeu et contexte métier	7
3.1 Présentation générale de League of Legends	8
3.2 Organisation d'une équipe	8
3.3 Ressources et objectifs importants	9
3.4 Draft et choix des champions	9
3.5 Intérêt métier d'un modèle de prédiction	9
4 Présentation du dataset	10
4.1 Source des données	10
4.2 Taille générale du dataset	10
4.3 Structure par position	11
4.4 Variable cible	11
4.5 Dataset final utilisé	11

5	Prétraitement des données	13
5.1	Rôle du prétraitement	13
5.2	Suppression des colonnes non utilisées	14
5.3	Filtrage des lignes d'équipe	14
5.4	Conversion numérique et valeurs manquantes	14
5.5	Préparation des variables	14
5.6	Séparation entraînement/test	15
5.7	Normalisation et pipelines	15
6	Feature engineering	16
6.1	Importance du feature engineering	16
6.2	Différence d'or et différence d'expérience	17
6.3	Différence de plaques	17
6.4	KDA collectif	18
6.5	Avantage de draft	18
6.6	Variables d'objectifs et side	18
7	Analyse exploratoire des données	19
7.1	Objectif de l'analyse exploratoire	20
7.2	Distribution de la variable cible	20
7.3	Corrélations avec la victoire	21
7.4	Combacks et throws	21
7.5	Synthèse de l'analyse exploratoire	22
8	Méthodologie Machine Learning	23
8.1	Formulation	23
8.2	Séparation des données	24
8.3	Normalisation	24
8.4	Pipelines	24
8.5	Métriques	24
8.6	Validation croisée et optimisation	24
9	Modèles utilisés	26
9.1	Vue générale	26
9.2	Logistic Regression	26

9.3	Random Forest	26
9.4	Gradient Boosting et AdaBoost	27
9.5	XGBoost	27
9.6	Voting et Stacking	27
10	Résultats expérimentaux et comparaison des modèles	28
10.1	Résultats des modèles de base	29
10.2	Optimisation des hyperparamètres	30
10.3	Résultats des modèles d'ensemble	30
10.4	Validation croisée	30
10.5	Importance des variables	31
10.6	Interprétation globale	31
11	Déploiement de l'application Flask	32
11.1	Objectif du déploiement	33
11.2	Architecture	33
11.3	Fonctionnement général	34
11.4	Calcul des features dans l'application	34
11.5	Prédiction et affichage	34
11.6	Limites de l'application	34
12	Limites, améliorations et conclusion générale	35
12.1	Importance de l'analyse critique	35
12.2	Limites principales	36
12.2.1	Séparation train/test	36
12.2.2	Timing des objectifs	36
12.2.3	Avantage de draft simplifié	36
12.2.4	Application Flask	36
12.3	Améliorations prioritaires	37
12.4	Conclusion générale	37
	Bibliographie et webographie	39
A	Annexes	40
A.1	Structure du projet	40
A.2	Liste finale des features	40

A.3	Correction recommandée du split train/test	41
A.4	Correction recommandée de la perspective Blue/Red dans Flask	41
A.5	Amélioration recommandée de l'avantage de draft	41
A.6	Fichier requirements recommandé	42
A.7	Liste finale des images à produire	42

Liste des tableaux

1	Plan recommandé des figures à insérer	x
2.1	Nature du problème traité	5
2.2	Comparaison des moments de prédiction	6
3.1	Rôles principaux dans League of Legends	8
4.1	Taille du dataset brut	10
4.2	Répartition des lignes par position	11
4.3	Codage de la variable cible	11
4.4	Évolution de la taille du dataset	12
6.1	Variables finales utilisées par le modèle	17
7.1	Distribution de la variable cible	20
7.2	Corrélations principales avec la victoire	21
7.3	Analyse des comebacks	21
8.1	Métriques d'évaluation	24
9.1	Modèles utilisés	26
10.1	Résultats des modèles de base	29
10.2	Résultats de l'optimisation	30
10.3	Comparaison des modèles d'ensemble	30
10.4	Validation croisée des modèles d'ensemble	31
10.5	Importance des variables selon Random Forest et XGBoost	31
11.1	Architecture de l'application	33
12.1	Améliorations prioritaires	37

Table des figures

1.1	Schéma global du système	3
3.1	Schéma simplifié de League of Legends	7
4.1	Structure joueur/team du dataset	10
5.1	Pipeline de nettoyage et de filtrage	13
6.1	Familles de variables finales	16
7.1	Heatmap des corrélations	19
7.2	Distribution de la différence d'or	20
8.1	Pipeline Machine Learning expérimental	23
10.1	Comparaison graphique des performances des modèles	28
10.2	Importance des variables du modèle	29
11.1	APPLICATION FLASK	32
11.2	-	33
12.1	Roadmap des améliorations du projet	35

Guide d'insertion des figures

Cette version V2 contient des emplacements de figures sous forme de cadres. Ces cadres ne sont pas des images finales : ils indiquent précisément quelle figure insérer, à quel endroit et dans quel objectif. Pour finaliser le rapport, il suffit de remplacer chaque cadre par une image réelle en utilisant

`includegraphics`. Les figures recommandées sont des captures du notebook, des graphiques générés par Python, des schémas simples créés dans draw.io/Canva/PowerPoint, et des captures de l'application Flask.

Table 1 – Plan recommandé des figures à insérer

N°	Emplacement	Figure recommandée
1	Introduction	Schéma général du système : dataset, prétraitement, modèle ML, application Flask.
2	Contexte métier	Schéma simplifié d'une partie de League of Legends : deux équipes, objectifs, Nexus.
3	Dataset	Capture ou tableau synthétique de la structure du CSV et des lignes joueur/équipe.
4	Prétraitement	Diagramme du pipeline de nettoyage : CSV brut, filtrage team, suppression NA, features finales.
5	Feature engineering	Schéma des familles de variables : économie, XP, draft, objectifs, combat.
6	Analyse exploratoire	Heatmap des corrélations entre variables et résultat.
7	Analyse exploratoire	Graphique de distribution de la différence d'or selon victoire/défaite.
8	Méthodologie ML	Schéma du pipeline d'apprentissage : split, scaler, modèles, évaluation.
9	Résultats	Bar chart comparant l'accuracy et le F1-score des modèles.

10	Résultats	Bar chart des importances de variables Random Forest ou XGBoost.
11	Déploiement	Capture d'écran de l'interface Flask.
12	Conclusion	Roadmap des améliorations prioritaires.

Chapitre 1

Introduction générale

L'évolution rapide des technologies numériques a profondément transformé la manière dont les données sont collectées, analysées et exploitées. Le Machine Learning occupe aujourd'hui une place centrale dans cette transformation, car il permet de construire des modèles capables d'apprendre à partir de données historiques afin de produire des prédictions, des classifications ou des recommandations. Ces techniques sont utilisées dans des domaines très variés : finance, santé, industrie, marketing, cybersécurité, sport et, plus récemment, esports.

L'esport constitue un terrain particulièrement intéressant pour l'analyse de données. Les compétitions de jeux vidéo génèrent naturellement une grande quantité d'informations structurées : résultats des matchs, performances des joueurs, compositions d'équipes, objectifs obtenus, durée des parties, différences de ressources et événements de jeu. Contrairement à certains sports traditionnels où certaines informations sont difficiles à mesurer précisément, les jeux compétitifs produisent des données détaillées et directement exploitables.

Parmi les jeux les plus importants dans l'univers de l'esport, *League of Legends* occupe une place majeure. Développé par Riot Games, ce jeu oppose deux équipes de cinq joueurs qui cherchent à détruire la base adverse. Le déroulement d'un match dépend de nombreux facteurs : les champions sélectionnés, les rôles des joueurs, les objectifs pris sur la carte, l'avance économique, la gestion de l'expérience, la coordination d'équipe et la capacité à convertir un avantage en victoire.

Dans un match de *League of Legends*, les premières minutes jouent un rôle important dans la construction de l'avantage d'une équipe. À quinze minutes de jeu, plusieurs indicateurs permettent déjà d'avoir une vision assez claire de l'état d'une partie. Une équipe peut avoir accumulé une avance en or, obtenu davantage d'expérience, détruit plus de plaques de tourelles, sécurisé certains objectifs neutres ou réalisé de meilleures performances collectives en combat. Ces éléments ne garantissent pas toujours la victoire, car des retournements de situation restent possibles, mais ils fournissent des signaux importants pour estimer les chances de victoire.

Ce projet vise à développer un système de prédiction capable d'estimer l'issue d'un match professionnel de *League of Legends* à partir de données disponibles autour de la quinzième minute. Il s'agit d'un problème de classification binaire : pour une équipe donnée, le modèle doit prédire si cette équipe va gagner ou perdre la partie. Le projet s'appuie sur des données issues d'Oracle's Elixir, une source spécialisée dans les statistiques de compétitions professionnelles de *League of Legends*.

Le travail réalisé ne se limite pas à l'entraînement d'un seul modèle. Il suit une démarche complète de projet Machine Learning : chargement et compréhension du dataset, nettoyage des données, sélection des lignes pertinentes, création de nouvelles variables, analyse exploratoire, comparaison de plusieurs algorithmes, optimisation des modèles, création d'un modèle d'ensemble, puis déploiement sous forme d'application web avec Flask.

L'intérêt de ce projet est double. D'un point de vue technique, il permet d'appliquer plusieurs notions importantes du Machine Learning, notamment le prétraitement, le feature engineering, la classification supervisée, l'évaluation des performances, les modèles d'ensemble et le déploiement. D'un point de vue applicatif, il montre comment des données issues d'un environnement compétitif réel peuvent être exploitées pour produire une estimation objective des chances de victoire.

Ce rapport présente l'ensemble des étapes suivies pour concevoir ce système prédictif. Il détaille d'abord la problématique et le contexte métier, puis explique le dataset utilisé, les étapes de prétraitement, les variables construites, l'analyse exploratoire, les modèles entraînés et les résultats obtenus. Enfin, il présente le déploiement de l'application Flask ainsi que les limites et améliorations possibles.

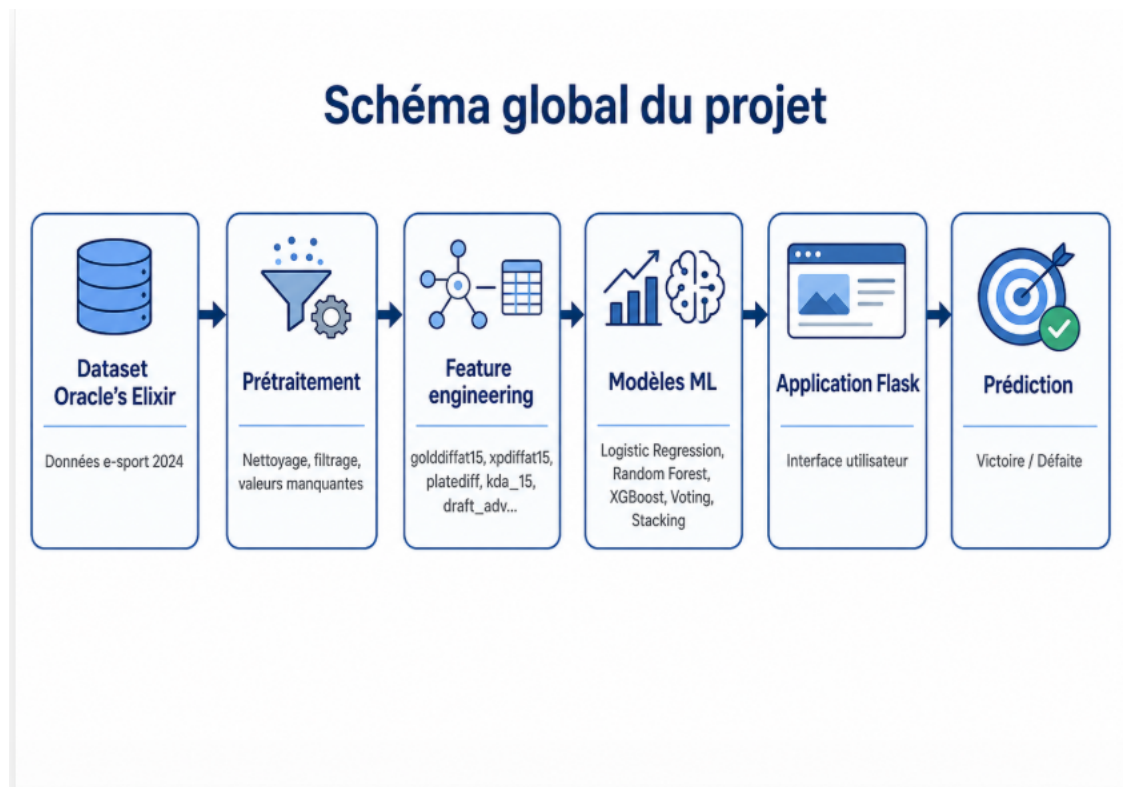


Figure 1.1 – Schéma global du système

Problématique et objectifs du projet

2.1 Contexte de la problématique

Dans un match professionnel de *League of Legends*, l'issue finale dépend d'un grand nombre de facteurs. Une partie n'est pas simplement déterminée par le nombre de kills ou par le choix d'un champion fort. Elle résulte d'une interaction complexe entre la stratégie de draft, la performance individuelle, la coordination collective, la gestion des ressources, le contrôle de la carte, la prise d'objectifs et la capacité à exploiter les erreurs adverses.

Cette complexité rend la prédiction du résultat particulièrement intéressante. Une équipe peut prendre un avantage important en début de partie grâce à une avance en or, une meilleure gestion des objectifs ou une domination sur certaines lanes. Cependant, cet avantage ne garantit pas automatiquement la victoire. Certaines équipes peuvent perdre malgré une avance significative, tandis que d'autres peuvent revenir grâce à une meilleure coordination, une composition plus efficace en fin de partie ou des erreurs de l'équipe adverse.

Le moment choisi pour effectuer la prédiction est la quinzième minute. Ce choix est pertinent car il correspond à une phase où la partie a déjà produit suffisamment d'informations pour permettre une analyse significative, tout en conservant une incertitude réelle sur le résultat final. La problématique principale du projet peut donc être formulée ainsi :

Peut-on prédire l'issue d'un match professionnel de League of Legends à partir des statistiques disponibles autour de la quinzième minute de jeu ?

2.2 Nature du problème Machine Learning

Le problème traité est un problème de classification supervisée binaire. Il est supervisé car chaque exemple du dataset possède une réponse connue : le résultat final de l'équipe.

Il est binaire car la variable cible ne possède que deux valeurs possibles : victoire ou défaite.

Table 2.1 – Nature du problème traité

Élément	Description
Type de problème	Classification supervisée
Nombre de classes	Deux classes
Classe positive	Victoire
Classe négative	Défaite
Unité d'analyse	Une équipe dans un match
Moment de prédiction	Environ 15 minutes de jeu
Sortie attendue	Victoire ou défaite avec probabilité associée

Chaque ligne du dataset final correspond à une équipe dans un match donné. Le modèle reçoit les informations relatives à cette équipe et doit prédire la classe correspondante. La classe positive représente la victoire, notée 1, tandis que la classe négative représente la défaite, notée 0.

2.3 Objectif général

L'objectif général du projet est de développer un système prédictif capable d'estimer les chances de victoire d'une équipe professionnelle de *League of Legends* à partir de données de match disponibles autour de la quinzième minute. Cet objectif implique la construction d'un pipeline complet allant de la donnée brute jusqu'à une application web fonctionnelle.

Le projet peut donc être résumé ainsi : concevoir, entraîner, évaluer et déployer un modèle de Machine Learning permettant de prédire l'issue d'un match professionnel de *League of Legends* à partir de statistiques intermédiaires.

2.4 Objectifs spécifiques

Les objectifs spécifiques sont les suivants :

- comprendre et exploiter le dataset Oracle's Elixir ;
- préparer les données pour l'apprentissage automatique ;
- construire des variables pertinentes représentant l'état d'une partie à quinze minutes ;

- comparer plusieurs modèles de classification ;
- évaluer les performances avec des métriques adaptées ;
- sauvegarder le modèle final ;
- intégrer le modèle dans une application web Flask ;
- analyser les limites et proposer des améliorations.

2.5 Intérêt du choix de la quinzième minute

Le choix de la quinzième minute n'est pas arbitraire. Avant le début du match, seules les informations de draft sont disponibles, ce qui rend la prédiction très difficile. Trop tard dans la partie, le résultat peut être presque déjà décidé. À quinze minutes, la partie a déjà produit des signaux importants : différences d'or et d'expérience, premiers objectifs, plaques, kills et dynamique générale.

Table 2.2 – Comparaison des moments de prédiction

Moment de prédiction	Limite principale
Avant le début du match	Peu d'informations disponibles, principalement la draft
Trop tard dans la partie	Le résultat peut être presque déjà décidé
À quinze minutes	Bon équilibre entre informations disponibles et incertitude

La quinzième minute constitue donc un moment pertinent pour construire un modèle prédictif. Elle permet d'utiliser des indicateurs significatifs tout en conservant une vraie incertitude sur le résultat final.

Présentation du jeu et contexte métier



Figure 3.1 – Schéma simplifié de League of Legends

3.1 Présentation générale de League of Legends

League of Legends est un jeu vidéo compétitif de type MOBA, c'est-à-dire *Multiplayer Online Battle Arena*. Deux équipes de cinq joueurs s'affrontent sur une carte appelée la Faille de l'invocateur. Chaque joueur contrôle un champion possédant des compétences, un rôle et un style de jeu particuliers. L'objectif final est de détruire le Nexus adverse, situé au cœur de la base ennemie.

Dans un contexte professionnel, *League of Legends* est un jeu hautement stratégique. Les équipes ne jouent pas simplement pour accumuler des éliminations. Elles doivent gérer la carte, contrôler les ressources, choisir les bons moments pour combattre, sécuriser les objectifs neutres, coordonner les déplacements et exploiter les erreurs adverses.

3.2 Organisation d'une équipe

Chaque équipe est composée de cinq joueurs, chacun occupant un rôle précis. Ces rôles structurent la stratégie de l'équipe et déterminent la position principale du joueur sur la carte.

Table 3.1 – Rôles principaux dans League of Legends

Rôle	Description
Top	Joueur situé sur la voie du haut. Il joue souvent des champions résistants, des duellistes ou des champions capables de gérer une voie isolée.
Jungle	Joueur évoluant dans la jungle. Il sécurise les monstres neutres, aide les autres voies et participe au contrôle de la carte.
Mid	Joueur situé sur la voie centrale. Il joue souvent des champions capables d'infliger beaucoup de dégâts ou d'influencer rapidement les autres zones.
Bot	Joueur principal de dégâts physiques à distance, souvent appelé ADC. Il devient généralement très important dans les combats d'équipe.
Support	Joueur qui accompagne principalement le bot. Il protège ses alliés, engage les combats, contrôle la vision et aide l'équipe.

3.3 Ressources et objectifs importants

L'or est l'une des ressources les plus importantes du jeu. Il permet d'acheter des objets qui augmentent la puissance des champions. Une équipe qui possède plus d'or que son adversaire dispose généralement d'un avantage matériel. L'expérience est également fondamentale, car elle permet aux champions de monter de niveau et d'améliorer leurs compétences.

Les tourelles et les plaques de tourelles jouent aussi un rôle important. Les plaques donnent de l'or en début de partie et indiquent une pression sur les structures adverses. Les objectifs neutres, comme le dragon et le héraut de la faille, permettent d'obtenir des bonus, d'ouvrir la carte et d'accélérer l'avantage.

3.4 Draft et choix des champions

Avant le début d'une partie, les équipes passent par une phase de draft. Elles sélectionnent les champions qui seront joués par leurs cinq joueurs. La draft est une dimension stratégique majeure, car elle influence le style de jeu, les forces en début ou fin de partie, la capacité à engager les combats, à défendre ou à contrôler les objectifs.

Dans ce projet, la draft est représentée par une variable appelée `draft_adv`. Cette variable cherche à estimer l'avantage théorique d'une équipe à partir des taux de victoire historiques de ses champions.

3.5 Intérêt métier d'un modèle de prédiction

Un modèle de prédiction de victoire peut être utile à plusieurs niveaux. Pour les analystes, il peut aider à comprendre les facteurs les plus liés à la victoire. Pour les commentateurs, il peut fournir une estimation objective des chances d'une équipe pendant un match. Pour les étudiants en Machine Learning, il constitue un exemple concret d'application de la classification supervisée à un domaine riche et dynamique.

Chapitre 4

Présentation du dataset

gameid	datacompleteness	playoffs	game	participantid	side	position	playername	teamname	champion	...	opp_csatis	golddiffstat	xpdiffstat	csdiffstat	killsat	assistsat	deathstat	opp_killsat	opp_assistsat	opp_deathstat
0	10660-10660_game_1	partial	0	1	Blue	top	Zika	LNG Esports	Aatrox	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	10660-10660_game_1	partial	0	1	2	Blue	jng	Weivel	LNG Esports	Maokai	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	10660-10660_game_1	partial	0	1	3	Blue	msd	Scout	LNG Esports	Orianna	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	10660-10660_game_1	partial	0	1	4	Blue	bot	GALA	LNG Esports	Kalista	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	10660-10660_game_1	partial	0	1	5	Blue	sup	Mark	LNG Esports	Senna	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Figure 4.1 – Structure joueur/team du dataset

4.1 Source des données

Le projet s'appuie sur un dataset issu d'Oracle's Elixir, une source spécialisée dans la collecte et la publication de données statistiques relatives aux compétitions professionnelles de *League of Legends*. Le fichier utilisé est :

2024_LoL_esports_match_data_from_OraclesElixir1.csv

Il contient des données relatives à des matchs professionnels joués durant l'année 2024. Le fichier est chargé avec `pandas` en utilisant le séparateur ;.

```
df = pd.read_csv(file_path, sep=';', low_memory=False)
```

4.2 Taille générale du dataset

Avant tout traitement, le dataset brut contient 12 276 lignes, 123 colonnes et 1 023 matchs uniques.

Table 4.1 – Taille du dataset brut

Élément	Valeur
Nombre total de lignes	12 276
Nombre total de colonnes	123
Nombre de matchs uniques	1 023

Chaque match est représenté par 12 lignes : cinq joueurs Blue, cinq joueurs Red, une ligne équipe Blue et une ligne équipe Red.

4.3 Structure par position

La colonne `position` permet de distinguer les rôles des joueurs et les lignes d'équipe. La répartition est équilibrée.

Table 4.2 – Répartition des lignes par position

Position	Nombre de lignes
top	2 046
jng	2 046
mid	2 046
bot	2 046
sup	2 046
team	2 046

Comme le projet cherche à prédire le résultat d'une équipe, les lignes `team` sont conservées pour l'entraînement du modèle.

4.4 Variable cible

La variable cible est `result`. Elle indique si l'équipe concernée a gagné ou perdu la partie.

Table 4.3 – Codage de la variable cible

Valeur	Signification
1	Victoire
0	Défaite

Après filtrage, le dataset final est parfaitement équilibré : 836 victoires et 836 défaites.

4.5 Dataset final utilisé

Après sélection des lignes d'équipe et suppression des lignes sans statistiques exploitables à quinze minutes, le dataset final contient 1 672 lignes, correspondant à 836 matchs.

Table 4.4 – Évolution de la taille du dataset

Étape	Nombre de lignes
Dataset brut	12 276
Lignes d'équipes	2 046
Après suppression des valeurs manquantes sur <code>golddiffat15</code>	1 672

Les variables finales utilisées sont : `golddiffat15`, `xpdiffat15`, `platediff`, `kda_15`, `draft_adv`, `side_encoded`, `firstblood`, `firstdragon`, `firsttherald` et `firsttower`.

Prétraitement des données

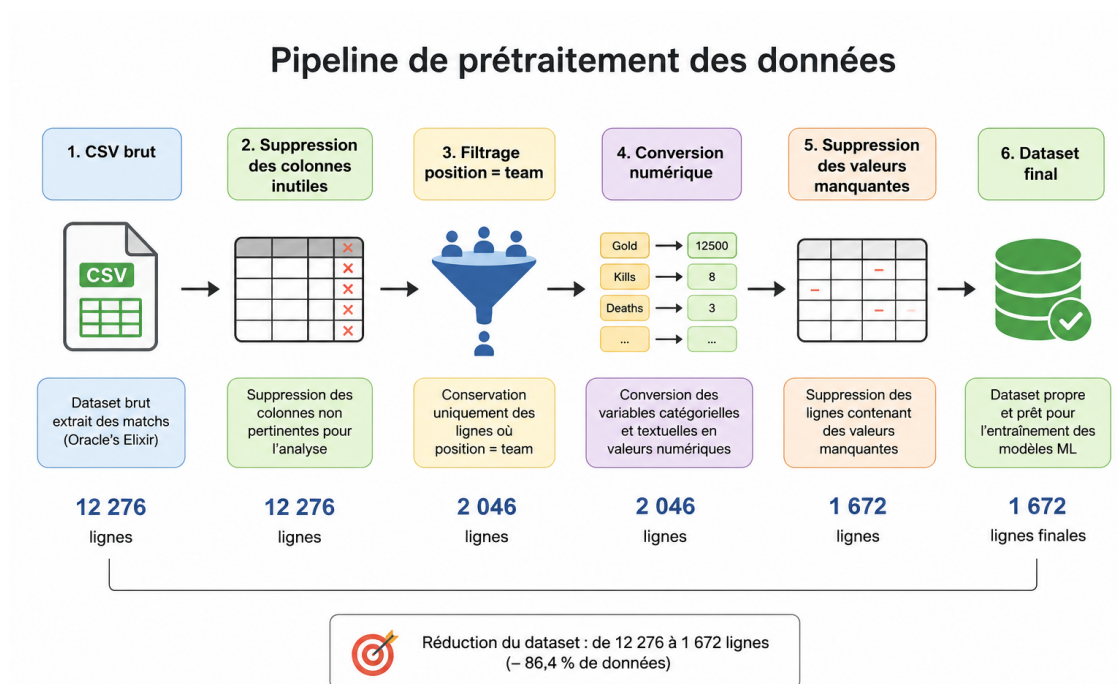


Figure 5.1 – Pipeline de nettoyage et de filtrage

5.1 Rôle du prétraitement

Le prétraitement transforme le fichier brut en un dataset propre, cohérent et adapté à l'apprentissage automatique. Dans ce projet, cette étape est particulièrement importante car le fichier contient à la fois des lignes joueurs et des lignes équipes, ainsi qu'un grand nombre de colonnes qui ne sont pas toutes utiles pour la prédiction.

5.2 Suppression des colonnes non utilisées

Certaines colonnes sont retirées : `url`, `league`, `year`, `split`, `date`, `patch`, `playerid` et `teamid`. Ces colonnes ne sont pas utilisées dans le modèle final. Leur suppression simplifie le dataset, même si certaines d'entre elles pourraient être utiles dans une version améliorée, notamment `patch` et `league`.

5.3 Filtrage des lignes d'équipe

Le modèle travaille au niveau de l'équipe. Le notebook filtre donc les lignes dont la position est égale à `team`.

```
teams = df[df['position'] == 'team'].copy()
```

Cette étape fait passer le dataset de 12 276 lignes à 2 046 lignes d'équipes.

5.4 Conversion numérique et valeurs manquantes

Certaines colonnes numériques peuvent être lues comme des chaînes de caractères. La variable `golddiffat15` est donc convertie en numérique :

```
teams['golddiffat15'] = pd.to_numeric(teams['golddiffat15'], errors='coerce')
teams = teams.dropna(subset=['golddiffat15'])
```

La suppression des lignes sans `golddiffat15` est justifiée car cette variable est centrale dans le projet. Après cette étape, 1 672 lignes restent exploitables.

5.5 Préparation des variables

Le prétraitement inclut aussi la création de variables supplémentaires. La différence de plaques est calculée par :

```
teams['platediff'] = teams['turretplates'] - teams['opp_turretplates']
```

Le KDA collectif à quinze minutes est calculé ainsi :

```
teams['kda_15'] = (teams['killsat15'] + teams['assistsat15']) / (teams['deathsat15'] + 1)
```

Le côté de la carte est encodé numériquement : Blue vaut 1 et Red vaut 0.

5.6 Séparation entraînement/test

Les données sont divisées en entraînement et test avec une proportion 80/20. Cette séparation est classique, mais elle présente une limite : elle est réalisée ligne par ligne. Comme deux lignes représentent le même match, il serait préférable d'utiliser une séparation par `gameid` avec `GroupShuffleSplit`.

5.7 Normalisation et pipelines

Les modèles sont intégrés dans des pipelines contenant `RobustScaler`. Cette normalisation est utile car les variables ont des échelles différentes. Par exemple, l'or et l'expérience peuvent atteindre plusieurs milliers, tandis que les objectifs sont binaires. `RobustScaler` est moins sensible aux valeurs extrêmes qu'une standardisation classique.

Feature engineering

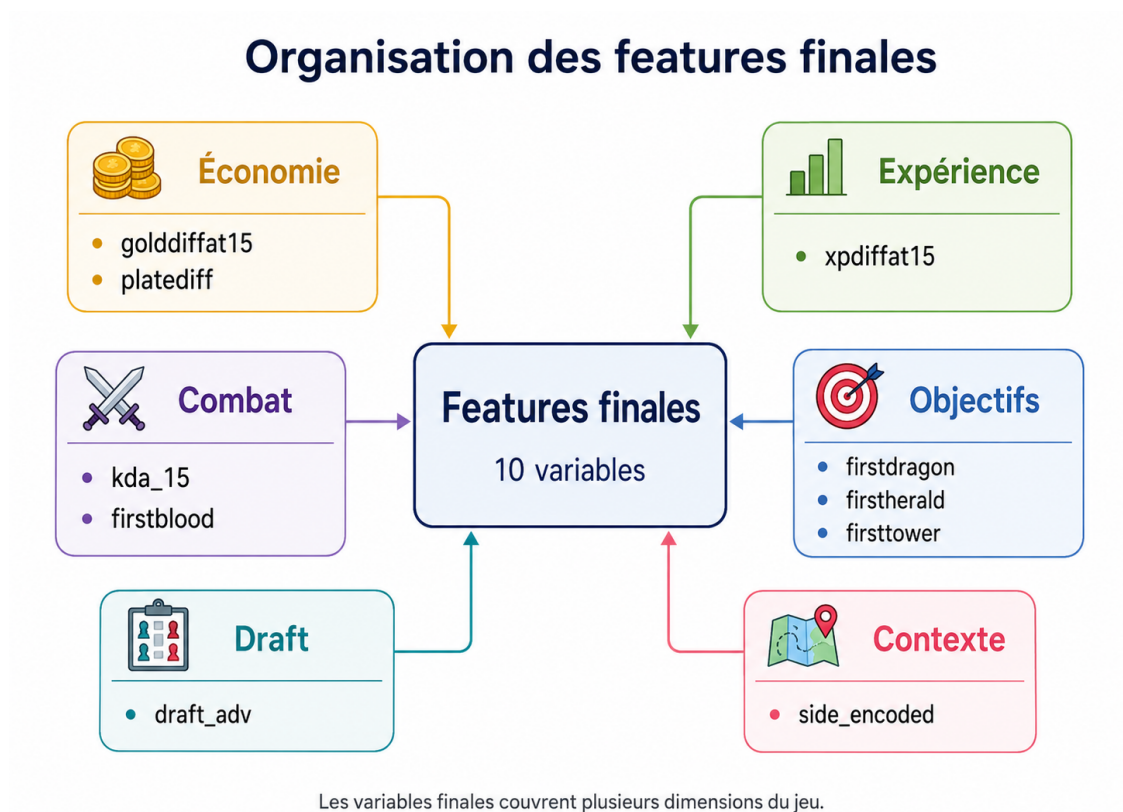


Figure 6.1 – Familles de variables finales

6.1 Importance du feature engineering

Le feature engineering consiste à créer ou transformer des variables afin de mieux représenter le problème étudié. Dans ce projet, il est central car le modèle doit comprendre l'état d'une équipe à quinze minutes à travers un nombre limité de variables pertinentes.

Table 6.1 – Variables finales utilisées par le modèle

Variable	Description
<code>golddiffat15</code>	Différence d'or entre l'équipe et l'adversaire à quinze minutes.
<code>xpdiffat15</code>	Différence d'expérience entre l'équipe et l'adversaire à quinze minutes.
<code>platediff</code>	Différence de plaques de tourelles détruites.
<code>kda_15</code>	Ratio KDA collectif à quinze minutes.
<code>draft_adv</code>	Avantage estimé de la composition selon les win rates historiques des champions.
<code>side_encoded</code>	Côté de la carte : Blue = 1, Red = 0.
<code>firstblood</code>	Indique si l'équipe a obtenu le premier kill.
<code>firstdragon</code>	Indique si l'équipe a obtenu le premier dragon.
<code>firsttherald</code>	Indique si l'équipe a obtenu le premier héraut.
<code>firsttower</code>	Indique si l'équipe a détruit la première tour.

6.2 Différence d'or et différence d'expérience

`golddiffat15` mesure l'avantage économique. Une valeur positive signifie que l'équipe possède plus d'or que son adversaire. Cette variable est fortement liée à la victoire car l'or permet d'acheter des objets, donc d'augmenter la puissance des champions.

`xpdiffat15` mesure l'avantage en expérience. Une équipe en avance en expérience possède souvent des champions plus haut niveau, avec de meilleures statistiques et des compétences plus avancées. L'expérience complète l'information donnée par l'or.

6.3 Différence de plaques

`platediff` est calculée par la formule :

$$\text{platediff} = \text{turretplates} - \text{opp_turretplates}$$

Cette variable représente la pression exercée sur les tourelles en début de partie. Une équipe qui prend plus de plaques obtient de l'or supplémentaire et montre une domination structurelle sur la carte.

6.4 KDA collectif

Le KDA collectif est calculé par :

$$\text{kda}_{15} = (\text{killsat15} + \text{assistsat15}) / (\text{deathsat15} + 1)$$

L'ajout de 1 au nombre de morts évite la division par zéro. Cette variable synthétise l'efficacité collective en combat. Elle ne suffit pas à elle seule à expliquer la victoire, mais elle apporte une information utile sur la qualité des échanges.

6.5 Avantage de draft

La variable `draft_adv` est construite à partir des taux de victoire historiques des champions dans le train set. Le principe est le suivant : calculer la moyenne des win rates des cinq champions d'une équipe, calculer la moyenne de l'équipe adverse, puis faire la différence.

$$\text{draft_adv} = \text{moyenne_équipe} - \text{moyenne_adversaire}$$

Cette variable intègre une dimension stratégique. Cependant, elle reste simplifiée car elle ne tient pas compte des synergies, des matchups, du patch ni du niveau des équipes.

6.6 Variables d'objectifs et side

Les variables `firstblood`, `firstdragon`, `firsttherald` et `firsttower` ajoutent des informations sur les premiers événements importants du match. Elles sont pertinentes mais doivent être utilisées avec prudence, car il faut s'assurer qu'elles sont réellement disponibles à quinze minutes.

`side_encoded` représente le côté de la carte. Son effet est plus faible que celui des ressources, mais il apporte un contexte supplémentaire.

Analyse exploratoire des données

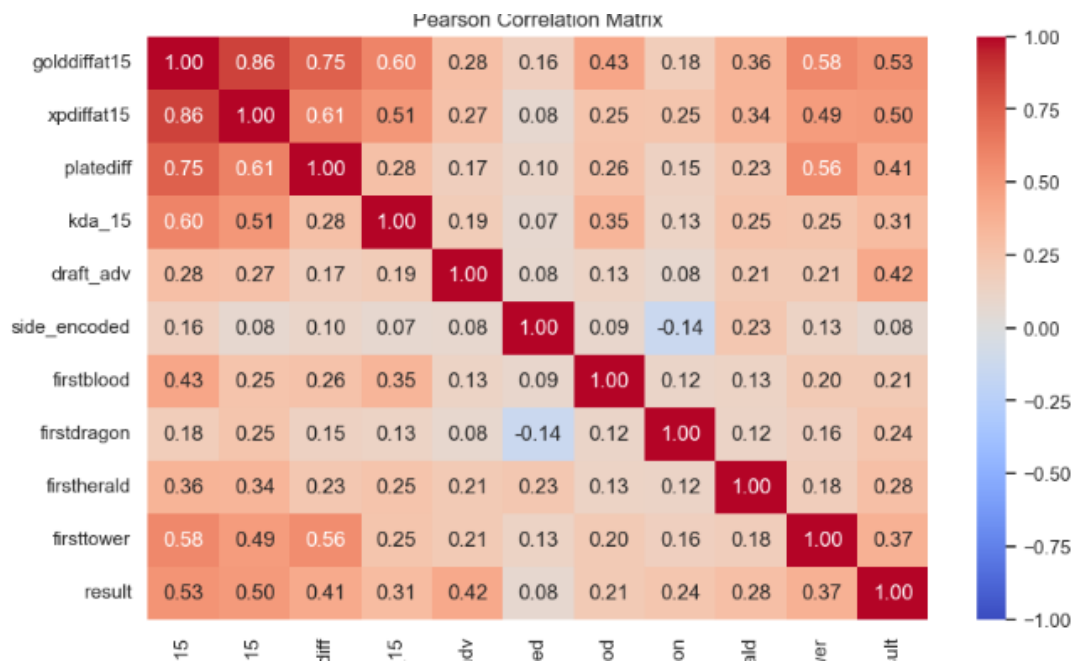


Figure 7.1 – Heatmap des corrélations

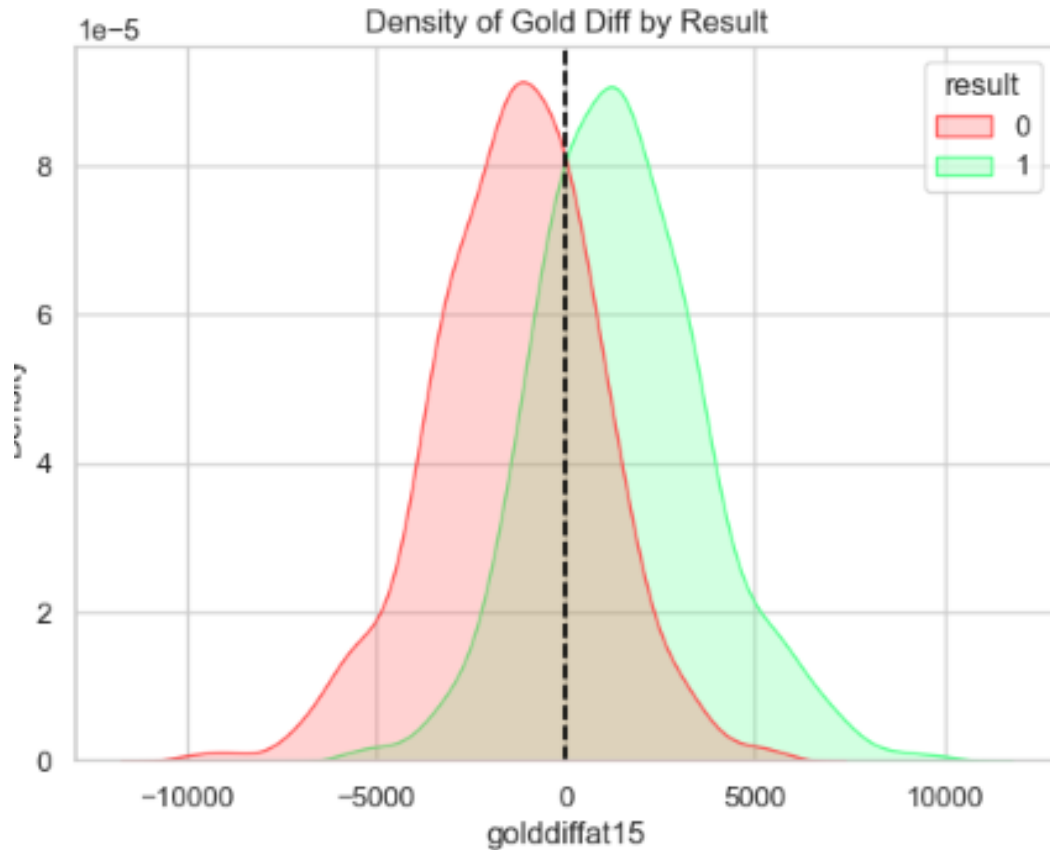


Figure 7.2 – Distribution de la différence d’or

7.1 Objectif de l’analyse exploratoire

L’analyse exploratoire permet de comprendre les relations entre les variables et la victoire avant l’entraînement des modèles. Elle permet d’identifier les variables les plus prometteuses, de vérifier l’équilibre de la cible et d’interpréter les tendances générales.

7.2 Distribution de la variable cible

Le dataset final est parfaitement équilibré : 836 victoires et 836 défaites. Cette situation est favorable car le modèle n’est pas biaisé vers une classe majoritaire.

Table 7.1 – Distribution de la variable cible

Classe	Signification	Nombre d’exemples
1	Victoire	836
0	Défaite	836

7.3 Corrélations avec la victoire

Les corrélations observées montrent que les variables les plus liées à la victoire sont la différence d'or, la différence d'expérience, l'avantage de draft et la différence de plaques.

Table 7.2 – Corrélations principales avec la victoire

Variable	Corrélation avec la victoire
golddiffat15	0.530
xpdiffat15	0.498
draft_adv	0.420
platediff	0.406
firsttower	0.369
kda_15	0.306
firstherald	0.283
firstdragon	0.236
firstblood	0.207
side_encoded	0.077

Ces résultats sont cohérents avec la logique du jeu. Les ressources économiques et l'expérience sont les indicateurs les plus forts. Les objectifs et le côté de la carte ont un effet plus limité.

7.4 Comebacks et throws

L'analyse des comebacks montre que 665 équipes étaient en retard en or à quinze minutes, et que 176 d'entre elles ont finalement gagné. Le taux de comeback est donc de 26,47%.

Table 7.3 – Analyse des comebacks

Élément	Valeur
Équipes en retard en or à 15 minutes	665
Équipes en retard ayant gagné	176
Taux de comeback	26,47%

Cette observation montre qu'une équipe en retard à quinze minutes peut encore gagner. Le modèle ne peut donc pas se limiter à une règle simple basée sur l'or.

L'analyse des throws identifie quatre cas où une équipe avait plus de 4 000 gold d'avance à quinze minutes mais a perdu. Cela confirme que même un avantage important ne garantit pas la victoire.

7.5 Synthèse de l'analyse exploratoire

L'analyse exploratoire confirme la pertinence des variables sélectionnées. L'or et l'expérience dominant, mais la draft, les plaques, le KDA et les objectifs ajoutent des informations complémentaires. Le côté de la carte a un effet faible. La présence de comebacks et de throws justifie l'utilisation d'un modèle de Machine Learning capable de combiner plusieurs signaux.

Méthodologie Machine Learning

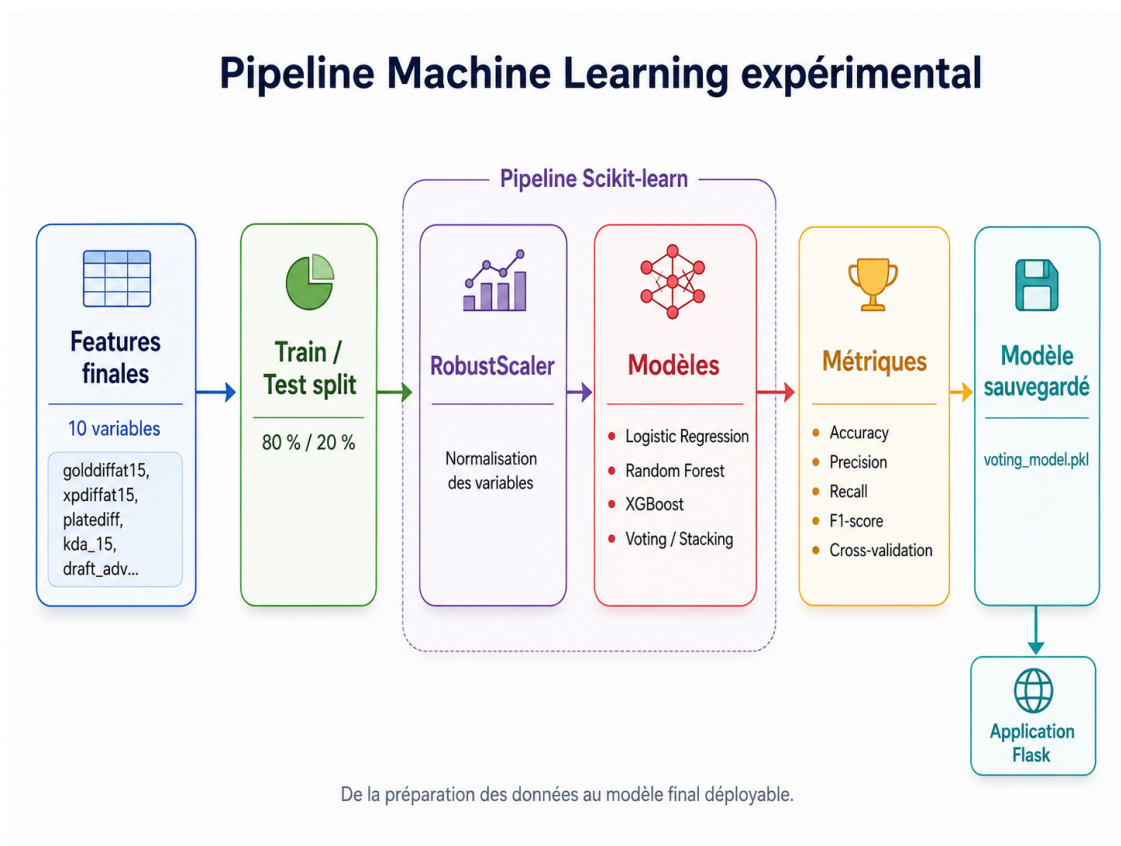


Figure 8.1 – Pipeline Machine Learning expérimental

8.1 Formulation

Le projet est formulé comme une classification supervisée binaire. Les entrées du modèle sont les dix variables décrivant l'état de l'équipe à quinze minutes, et la sortie est la variable `result`.

8.2 Séparation des données

Les données sont divisées en 80% pour l'entraînement et 20% pour le test. Cette séparation permet d'évaluer la généralisation du modèle. Toutefois, une séparation par `gameid` serait plus rigoureuse car les deux lignes d'un même match sont dépendantes.

8.3 Normalisation

Les variables ont des échelles différentes. Le projet utilise `RobustScaler`, qui normalise les données à partir de la médiane et de l'écart interquartile. Ce choix est adapté aux variables pouvant contenir des valeurs extrêmes, comme les différences d'or ou d'expérience.

8.4 Pipelines

Les modèles sont intégrés dans des pipelines Scikit-learn. Un pipeline permet de regrouper la normalisation et le modèle dans une même structure. Cela réduit les erreurs, facilite la validation croisée et rend le modèle plus facile à sauvegarder.

8.5 Métriques

Les modèles sont évalués avec plusieurs métriques.

Table 8.1 – Métriques d'évaluation

Métrique	Rôle
Accuracy	Proportion totale de prédictions correctes.
Precision	Fiabilité des prédictions positives.
Recall	Capacité à détecter les vraies victoires.
F1-score	Équilibre entre précision et rappel.
Cross-validation	Estimation de la stabilité du modèle.

8.6 Validation croisée et optimisation

La validation croisée permet de mesurer la stabilité des modèles sur plusieurs sous-ensembles. L'optimisation des hyperparamètres est utilisée pour Random Forest,

XGBoost et Logistic Regression afin d'améliorer leur performance et de limiter le surapprentissage.

Modèles utilisés

9.1 Vue générale

Plusieurs modèles sont entraînés afin de comparer différentes approches.

Table 9.1 – Modèles utilisés

Modèle	Famille	Rôle
Logistic Regression	Linéaire	Modèle simple de référence
Random Forest	Ensemble d'arbres	Relations non linéaires
Gradient Boosting	Boosting	Correction progressive des erreurs
AdaBoost	Boosting adaptatif	Pondération des exemples difficiles
XGBoost	Boosting optimisé	Modèle performant sur données tabulaires
Voting Classifier	Ensemble	Combinaison par vote souple
Stacking Classifier	Ensemble	Combinaison par méta-modèle

9.2 Logistic Regression

La régression logistique est un modèle simple et interprétable. Elle apprend une combinaison linéaire des variables, puis transforme cette combinaison en probabilité. Dans ce projet, elle sert de modèle de référence. Ses bons résultats montrent que les variables principales expliquent déjà fortement la victoire.

9.3 Random Forest

Random Forest combine plusieurs arbres de décision. Chaque arbre apprend des règles sur les données, puis la forêt combine les prédictions. Ce modèle est capable de capturer des relations non linéaires et fournit des importances de variables. Il est

particulièrement utile pour comprendre quelles features influencent le plus la prédiction.

9.4 Gradient Boosting et AdaBoost

Gradient Boosting entraîne des modèles successifs qui corrigent les erreurs précédentes. AdaBoost augmente le poids des exemples mal classés pour forcer les modèles suivants à se concentrer sur les cas difficiles. Ces méthodes permettent de tester des approches séquentielles de boosting.

9.5 XGBoost

XGBoost est une version optimisée du Gradient Boosting. Il est souvent performant sur les données tabulaires grâce à sa régularisation, sa gestion de la complexité et ses nombreux hyperparamètres. Dans ce projet, il permet de tester un modèle avancé capable d'apprendre des interactions complexes.

9.6 Voting et Stacking

Le Voting Classifier combine Random Forest, XGBoost et Logistic Regression par vote souple. Il utilise les probabilités prédites par les modèles de base. Le Stacking Classifier va plus loin : il entraîne un méta-modèle qui apprend à combiner les prédictions des modèles de base.

Le modèle final utilisé dans l'application Flask est le Voting Classifier, car il offre un bon compromis entre performance, stabilité et simplicité de déploiement.

Résultats expérimentaux et comparaison des modèles

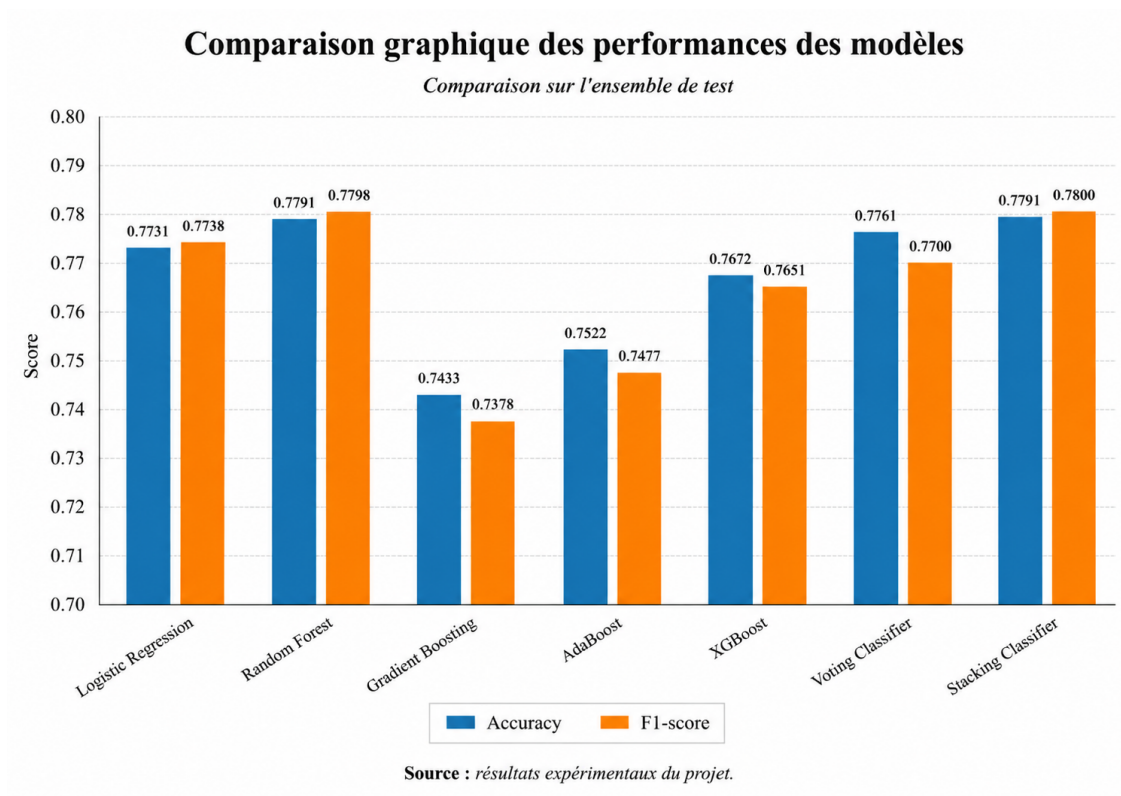


Figure 10.1 – Comparaison graphique des performances des modèles

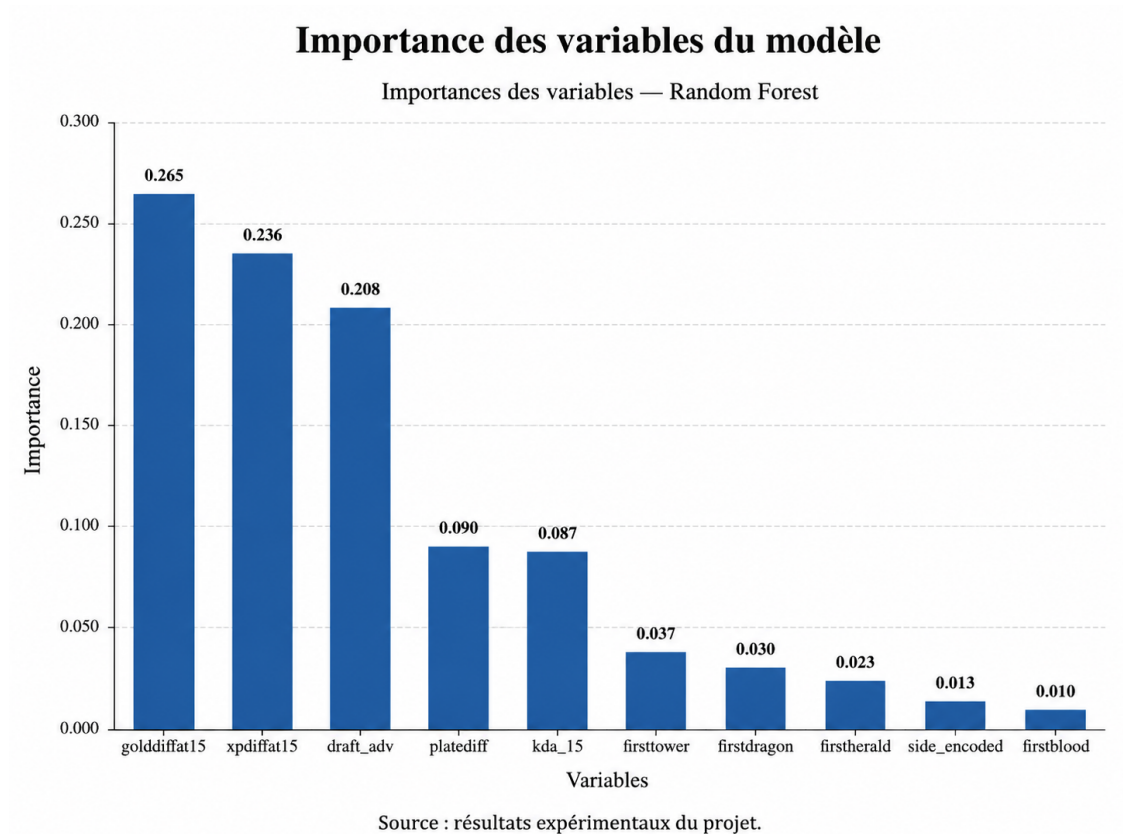


Figure 10.2 – Importance des variables du modèle

10.1 Résultats des modèles de base

Les performances des modèles de base sont présentées dans le tableau suivant.

Table 10.1 – Résultats des modèles de base

Modèle	Accuracy	F1-score
Logistic Regression	0.7731	0.7738
Random Forest	0.7791	0.7798
Gradient Boosting	0.7433	0.7378
AdaBoost	0.7522	0.7477
XGBoost	0.7672	0.7651

Random Forest obtient le meilleur résultat parmi les modèles de base. Cependant, la régression logistique reste très proche, ce qui montre que les relations entre les variables et la victoire sont en grande partie capturables par un modèle simple.

10.2 Optimisation des hyperparamètres

Les meilleurs paramètres obtenus sont résumés ci-dessous.

Table 10.2 – Résultats de l’optimisation

Modèle	Meilleurs paramètres	Score CV
Random Forest	n_estimators=50, max_features=sqrt, min_samples_split=2	max_depth=8, 0.7651
XGBoost	n_estimators=100, learning_rate=0.05, colsample_bytree=0.7	max_depth=3, 0.7696 subsample=0.9,
Logistic Regression	C=0.1, penalty=l2, solver=liblinear	0.7726

La régression logistique optimisée obtient le meilleur score de validation croisée parmi ces trois modèles, ce qui confirme l’importance des variables choisies.

10.3 Résultats des modèles d’ensemble

Le Voting Classifier obtient une accuracy de 0.7761. Le Stacking Classifier obtient une accuracy de 0.7791. L’écart est très faible, ce qui ne permet pas d’affirmer que le stacking est clairement supérieur.

Table 10.3 – Comparaison des modèles d’ensemble

Modèle	Accuracy
Voting Classifier	0.7761
Stacking Classifier	0.7791

10.4 Validation croisée

Les résultats de validation croisée confirment une performance moyenne autour de 77%.

Table 10.4 – Validation croisée des modèles d'ensemble

Modèle	Moyenne CV	Écart-type
Voting Classifier	0.7711	0.0320
Stacking Classifier	0.7719	0.0323

10.5 Importance des variables

Random Forest et XGBoost confirment que les variables les plus importantes sont la différence d'or, la différence d'expérience et l'avantage de draft.

Table 10.5 – Importance des variables selon Random Forest et XGBoost

Variable	Random Forest	XGBoost
golddiffat15	0.265	0.296
xpdiffat15	0.236	0.181
draft_adv	0.208	0.165
platediff	0.090	0.054
kda_15	0.087	0.046
firsttower	0.037	0.039
firstdragon	0.030	0.080
firsttherald	0.023	0.089
side_encoded	0.013	0.022
firstblood	0.010	0.029

10.6 Interprétation globale

Les performances autour de 77% à 78% sont encourageantes. Elles montrent que les statistiques à quinze minutes sont prédictives. Cependant, elles doivent être interprétées avec prudence à cause du risque de fuite d'information lié au split par lignes et à certaines variables d'objectifs.

Déploiement de l'application Flask

The screenshot shows a web application titled "Esports Match Predictor" with a trophy icon. It features a dark theme. At the top, a question "Which side are YOU playing on?" is followed by a dropdown menu currently set to "Blue Side". Below this is a section header "Step 1: Team Rosters (Champions & Current Levels)". The main area is divided into two panels: "Blue Team" on the left and "Red Team" on the right. Each panel contains a table with five rows representing game roles: TOP, JUNGLE, MID, ADC, and SUPPORT. Each row has a role label, a dropdown menu for selecting a champion (all currently show "Champion..."), and a text input for the current level (all currently show "10").

Blue Team		
TOP	Champion...	10
JUNGLE	Champion...	10
MID	Champion...	10
ADC	Champion...	10
SUPPORT	Champion...	10

Red Team		
TOP	Champion...	10
JUNGLE	Champion...	10
MID	Champion...	10
ADC	Champion...	10
SUPPORT	Champion...	10

Figure 11.1 – APPLICATION FLASK

Step 2: 15-Minute Macro State

Team Gold Difference
e.g. 1500 (Ahead) or -500 (Behind)

Turret Plate Differential
e.g. 2 (Ahead) or -1 (Behind)

Overall Team KDA Ratio ((K+A)/D)
e.g. 3.5

Step 3: Objectives Control (at 15 min)

First Blood
Blue Team

First Dragon
Blue Team

First Herald
Blue Team

First Tower
Blue Team

Execute Prediction Protocol

Figure 11.2 – -

11.1 Objectif du déploiement

Le déploiement permet de transformer le modèle entraîné en un outil utilisable. Dans ce projet, l'application Flask permet à l'utilisateur de saisir les informations d'un match et d'obtenir une prédiction de victoire ou de défaite.

11.2 Architecture

L'application repose sur trois éléments principaux :

Table 11.1 – Architecture de l'application

Élément	Rôle
<code>app.py</code>	Backend Flask, chargement du modèle et prédiction
<code>templates/index.html</code>	Interface utilisateur
<code>models/voting_model.pkl</code>	Modèle final sauvegardé
<code>models/champ_win_rates.pkl</code>	Taux de victoire historiques des champions

11.3 Fonctionnement général

Le flux de l'application est le suivant :

Utilisateur → Formulaire HTML → Flask → Calcul des features → Modèle ML →
Prédiction → Affichage du résultat

L'utilisateur sélectionne les champions, indique le côté de l'équipe, saisit les niveaux, la différence d'or, la différence de plaques, le KDA et les objectifs. L'application transforme ces valeurs en dix variables attendues par le modèle.

11.4 Calcul des features dans l'application

L'application reconstruit les mêmes variables que celles utilisées pendant l'entraînement : `golddiffat15`, `xpdiffat15`, `platediff`, `kda_15`, `draft_adv`, `side_encoded`, `firstblood`, `firstdragon`, `firsttherald` et `firsttower`.

La variable `xpdiffat15` est approximée à partir des niveaux des joueurs :

$$\text{xpdiffat15} = \text{level_diff} * 850$$

Cette approximation rend l'application plus simple à utiliser, mais elle crée un écart avec la donnée originale d'entraînement.

11.5 Prédiction et affichage

Le modèle est chargé avec `joblib`. La méthode `predict()` donne la classe finale et `predict_proba()` donne les probabilités associées. L'application affiche ensuite la prédiction et le niveau de confiance.

11.6 Limites de l'application

Plusieurs limites doivent être mentionnées. Les objectifs doivent être correctement convertis selon le côté analysé. Si l'utilisateur choisit Red Side, un objectif obtenu par Red doit être transmis au modèle comme une valeur favorable. L'application devrait aussi proposer une option « objectif non encore obtenu », car à quinze minutes certains objectifs peuvent ne pas avoir été pris.

Une autre amélioration serait de demander séparément kills, deaths et assists afin de calculer automatiquement le KDA, au lieu de demander à l'utilisateur de saisir directement le ratio.

Limites, améliorations et conclusion générale

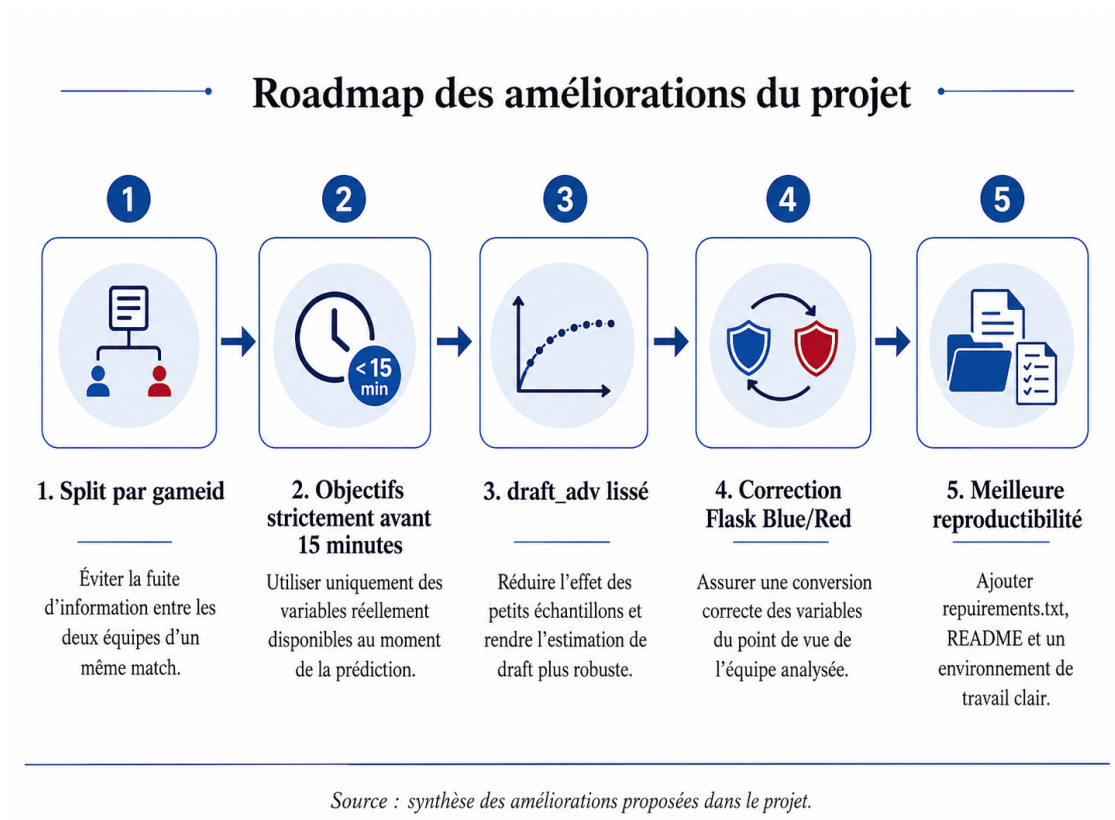


Figure 12.1 – Roadmap des améliorations du projet

12.1 Importance de l'analyse critique

Un projet de Machine Learning ne doit pas seulement présenter des résultats. Il doit aussi analyser ses limites. Cette étape permet d'interpréter les performances avec rigueur et de proposer des améliorations réalistes.

12.2 Limites principales

12.2.1 Séparation train/test

La principale limite concerne la séparation des données. Le split est réalisé par lignes, alors que chaque match possède deux lignes d'équipes. Il existe donc un risque que la ligne Blue d'un match soit dans l'entraînement et la ligne Red du même match dans le test. Cela peut créer une fuite d'information et surestimer les performances.

La correction recommandée est d'utiliser `GroupShuffleSplit` ou `GroupKFold` avec `gameid` comme groupe.

12.2.2 Timing des objectifs

Les variables `firstdragon`, `firstherald` et `firsttower` peuvent poser problème si elles indiquent l'objectif obtenu sur toute la partie et non uniquement avant quinze minutes. Si l'objectif a été obtenu après quinze minutes, il s'agit d'une information future.

12.2.3 Avantage de draft simplifié

`draft_adv` repose sur des taux de victoire historiques simples. Certains champions ont peu d'exemples, ce qui peut produire des valeurs extrêmes de 0% ou 100%. Une amélioration serait d'utiliser un lissage statistique et de calculer les taux par rôle et par patch.

12.2.4 Application Flask

L'application approxime l'expérience à partir des niveaux et doit mieux gérer la perspective Blue/Red pour les objectifs. Elle devrait aussi proposer l'option « objectif non pris ».

12.3 Améliorations prioritaires

Table 12.1 – Améliorations prioritaires

Priorité	Amélioration	Objectif
1	Split par <code>gameid</code>	Éviter la fuite d'information entre train et test
2	Vérifier le timing des objectifs	Garantir une prédiction réellement basée sur 15 minutes
3	Corriger la perspective Red Side	Rendre l'application fiable pour les deux côtés
4	Lisser <code>draft_adv</code>	Réduire l'effet des petits échantillons
5	Ajouter <code>requirements.txt</code> et README	Améliorer la reproductibilité

12.4 Conclusion générale

Ce projet avait pour objectif de concevoir un système capable de prédire l'issue d'un match professionnel de *League of Legends* à partir de données disponibles autour de la quinzième minute. Pour atteindre cet objectif, une démarche complète de Machine Learning a été mise en place, depuis l'exploitation du dataset Oracle's Elixir jusqu'au déploiement d'une application web avec Flask.

Le dataset utilisé contient des données professionnelles de l'année 2024. Après filtrage et nettoyage, les lignes d'équipes exploitables ont été conservées afin de formuler le problème comme une classification supervisée binaire. La variable cible utilisée est `result`, indiquant si une équipe a gagné ou perdu la partie.

Un travail important de feature engineering a été réalisé. Les variables retenues représentent plusieurs dimensions essentielles du jeu : la différence d'or, la différence d'expérience, la différence de plaques, le KDA collectif, l'avantage de draft, le côté de la carte et les premiers objectifs. Ces variables permettent de décrire l'état d'une équipe à quinze minutes de manière synthétique mais pertinente.

Plusieurs modèles ont été entraînés et comparés : Logistic Regression, Random Forest, Gradient Boosting, AdaBoost, XGBoost, Voting Classifier et Stacking Classifier. Les résultats montrent que les modèles atteignent environ 77% à 78% d'accuracy selon la configuration utilisée. Les meilleures performances sont obtenues par Random Forest et Stacking Classifier, tandis que le Voting Classifier est retenu pour le déploiement dans

l'application Flask.

L'analyse des importances de variables confirme que les facteurs les plus déterminants sont la différence d'or, la différence d'expérience et l'avantage de draft. Les plaques, le KDA et certains objectifs apportent également des informations utiles. Le côté de la carte et le first blood ont un impact plus limité.

Malgré ses limites, ce projet constitue une preuve de concept solide. Il montre que les données de *League of Legends* peuvent être exploitées efficacement par des méthodes de Machine Learning afin d'estimer l'issue d'un match à partir d'un état intermédiaire. Les performances obtenues sont encourageantes, mais une validation plus stricte et des features plus avancées permettraient d'améliorer la fiabilité du système.

Bibliographie

- [1] Oracle's Elixir. *League of Legends Esports Match Data*. Données statistiques professionnelles utilisées pour l'analyse des matchs de League of Legends.
- [2] Scikit-learn Documentation. *Machine Learning in Python*. Documentation officielle des modèles, pipelines, scalers, validation croisée et métriques.
- [3] XGBoost Documentation. *Extreme Gradient Boosting*. Documentation officielle de la bibliothèque XGBoost.
- [4] Flask Documentation. *Web Development, one drop at a time*. Documentation officielle du framework Flask.
- [5] Pandas Documentation. *Python Data Analysis Library*. Documentation officielle pour la manipulation de données tabulaires.
- [6] Riot Games. *League of Legends*. Documentation et informations générales sur le jeu, ses objectifs et son fonctionnement compétitif.
- [7] Joblib Documentation. *Lightweight pipelining in Python*. Documentation utilisée pour la sauvegarde et le chargement des modèles.

Annexe A

Annexes

A.1 Structure du projet

```
lol predict/  
|-- 2024_LoL_esports_match_data_from_OraclesElixir1.csv  
|-- LoL_15Min_Predictive_Ensemble.ipynb  
|-- app.py  
|-- models/  
| |-- champ_win_rates.pkl  
| '-- voting_model.pkl  
|-- templates/  
| '-- index.html
```

A.2 Liste finale des features

```
features = [  
    'golddiffat15',  
    'xpdiffat15',  
    'platediff',  
    'kda_15',  
    'draft_adv',  
    'side_encoded',  
    'firstblood',  
    'firstdragon',  
    'firsttherald',  
    'firsttower'  
]
```

A.3 Correction recommandée du split train/test

La correction suivante permet de séparer les données par match afin d'éviter que les deux équipes d'une même partie soient réparties entre l'entraînement et le test.

```
from sklearn.model_selection import GroupShuffleSplit

groups = teams['gameid']
gss = GroupShuffleSplit(test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(teams, teams['result'], groups=groups))

train_teams = teams.iloc[train_idx].copy()
test_teams = teams.iloc[test_idx].copy()
```

A.4 Correction recommandée de la perspective Blue/-Red dans Flask

Lorsqu'une variable d'objectif est saisie sous la forme Blue ou Red, elle doit être convertie du point de vue de l'équipe analysée.

```
def objective_from_selected_team(objective_winner, selected_side):
    # objective_winner: 'blue' or 'red'
    # selected_side: 'blue' or 'red'
    return 1 if objective_winner == selected_side else 0

firstblood = objective_from_selected_team(firstblood_winner, selected_side)
firstdragon = objective_from_selected_team(firstdragon_winner, selected_side)
firsttherald = objective_from_selected_team(firsttherald_winner, selected_side)
firsttower = objective_from_selected_team(firsttower_winner, selected_side)
```

A.5 Amélioration recommandée de l'avantage de draft

Un lissage statistique permet d'éviter qu'un champion joué une seule fois obtienne artificiellement un taux de victoire de 0% ou 100%.

```
global_wr = train_players['result'].mean()
prior_games = 10
```

```

champ_stats = train_players.groupby('champion')['result'].agg(['sum', 'count'
    ])
champ_stats['smoothed_wr'] = (
    champ_stats['sum'] + prior_games * global_wr
) / (champ_stats['count'] + prior_games)

champ_win_rates = champ_stats['smoothed_wr'].to_dict()

```

A.6 Fichier requirements recommandé

```

flask
pandas
numpy
scikit-learn
xgboost
joblib
matplotlib
seaborn
jupyter

```

A.7 Liste finale des images à produire

N°	Nom recommandé du fichier	Description
1	architecture_generale.png	Schéma global du système.
2	contexte_lol.png	Schéma simplifié de League of Legends.
3	structure_dataset.png	Structure joueur/team du dataset.
4	preprocessing_pipeline.png	Pipeline de nettoyage et filtrage.
5	features_families.png	Familles de variables finales.
6	correlation_heatmap.png	Heatmap des corrélations.
7	gold_diff_distribution.png	Distribution de la différence d'or.
8	ml_pipeline.png	Pipeline Machine Learning.
9	model_comparison.png	Comparaison des modèles.
10	feature_importance.png	Importance des variables.
11	flask_interface.png	Capture de l'application Flask.
12	improvement_roadmap.png	Roadmap d'amélioration.